Transactions & ACID

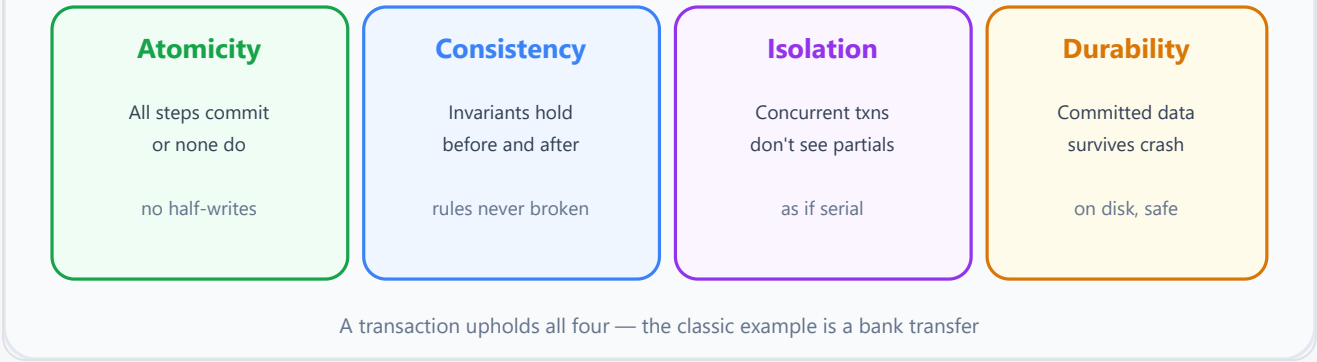
Why it matters

Some operations must happen all-or-nothing. Transfer money between two accounts: debit one, credit the other. If the process crashes after the debit but before the credit, money has vanished into thin air and your users (and regulators) will not forgive you. Place an order: decrement inventory, create the order, charge the card — if any step fails, none should stick. The mechanism that makes "all-or-nothing" real is a **transaction**, and the guarantees it provides are summarised by the acronym **ACID**. This is bedrock backend knowledge and one of the most reliably asked interview topics, because getting it wrong causes the most damaging class of bugs: silent data corruption.

What makes this topic sharp for interviews is the modern twist: transactions are not just a SQL thing anymore. MongoDB has supported **multi-document ACID transactions** since 4.0 (via sessions), so "does NoSQL have transactions?" is a live question with a nuanced answer. Knowing when you actually need a transaction — and, just as importantly, when a good document model means you do not — is what distinguishes solid engineers.

The concept, simply

A **transaction** groups several database operations into a single unit that either *fully commits* or *fully rolls back*. You mark a beginning, do your writes, and then either `COMMIT` (make them all permanent together) or, if anything goes wrong, `ROLLBACK` (undo everything as if none of it happened). The database guarantees no in-between state is ever visible or persisted.



ACID is the four-part promise a transaction upholds: all-or-nothing, valid state, isolated from other transactions, and durable once committed.

ACID names the four guarantees. *Atomicity*: all steps or none. *Consistency*: the database moves from one valid state to another, never breaking its rules. *Isolation*: concurrent transactions do not see each other's half-finished work. *Durability*: once committed, the data survives a crash or power loss.

How it works (deep)

Under the hood, a relational database implements atomicity and durability with a **write-ahead log (WAL)**: changes are recorded to a durable log before being applied, so on crash recovery the database can replay committed transactions and discard uncommitted ones. A `COMMIT` is not acknowledged until the log is safely on disk — that is durability. A `ROLLBACK` (or a crash) discards the pending changes — that is atomicity.



Either both writes commit together, or an error before commit triggers a rollback that undoes everything.

repeatable read (a row changes value between two reads in the same transaction), and a *phantom read* (new rows appear matching your `WHERE` between reads). SQL defines **isolation levels** — Read Uncommitted, Read Committed, Repeatable Read, Serializable — that trade performance for stronger guarantees, each ruling out more anomalies. Most databases default to Read Committed (Postgres) or Repeatable Read (MySQL InnoDB); Serializable is the strictest, behaving as if transactions ran one at a time. Naming an anomaly and the level that prevents it is a strong interview answer.

In **MongoDB**, a single-document write is *already atomic* — an update to one document, including its nested arrays and sub-documents, either fully applies or not at all. This is a big deal: if you model well and keep data that changes together in one document, **you often do not need a multi-document transaction at all**. When you genuinely must update several documents atomically (money moving between two separate account documents), Mongo provides multi-document transactions through a **session**: you `startSession()`, `startTransaction()`, do your writes passing the session, then `commitTransaction()` or `abortTransaction()`. These require a replica set or sharded cluster (not a standalone `mongod`), and they carry more overhead than single-document writes, so you reach for them deliberately.

When do you actually need a transaction? When a single logical operation spans multiple rows/documents and a partial result would be invalid or unsafe: financial transfers, inventory-plus-order, anything with an invariant across records ("total debits equal total credits"). When it is a single-document or single-row change, the database already gives you atomicity for free. Over-using transactions adds locking and contention; under-using them corrupts data. Knowing the line is the skill.

Code examples

A SQL transaction in node-postgres — the canonical money transfer:

JAVASCRIPT

Copy

```
async function transfer(fromId, toId, amount) {
  const client = await pool.connect();
  try {
```

```
    'SELECT balance FROM accounts WHERE id = $1 FOR UPDATE', [fromId]
  );                                    // FOR UPDATE locks the row
  if (rows[0].balance < amount) throw new Error('Insufficient funds');

  await client.query('UPDATE accounts SET balance = balance - $1 WHERE i
  await client.query('UPDATE accounts SET balance = balance + $1 WHERE i

  await client.query('COMMIT');         // both updates land together
} catch (err) {
  await client.query('ROLLBACK');       // any failure undoes everything
  throw err;
} finally {
  client.release();
}
}
```

The same all-or-nothing guarantee in MongoDB via a session:

JAVASCRIPT Copy

```
async function transfer(fromId, toId, amount) {
  const session = await mongoose.startSession();
  try {
    session.startTransaction();

    const from = await Account.findById(fromId).session(session);
    if (from.balance < amount) throw new Error('Insufficient funds');

    await Account.updateOne({ _id: fromId }, { $inc: { balance: -amount }
    await Account.updateOne({ _id: toId },   { $inc: { balance:  amount }

    await session.commitTransaction();   // atomic across both documents
  } catch (err) {
    await session.abortTransaction();    // rollback
    throw err;
  } finally {
    session.endSession();                // always end the session
  }
```

Often you do *not* need a transaction — single-document atomicity is enough:

JAVASCRIPT Copy

```
// Atomic decrement guarded by a condition, in ONE document operation.
// No transaction needed: the whole update applies or it does not.
const res = await Inventory.updateOne(
  { _id: productId, stock: { $gte: qty } }, // only if enough stock
  { $inc: { stock: -qty } }
);
if (res.modifiedCount === 0) throw new Error('Out of stock');
// This also avoids a race: the condition and the decrement are one atomic
```

Postgres offers the same conditional-atomic trick without an explicit transaction for a single statement:

SQL Copy

```
-- Single-statement atomicity: the WHERE guard and the update are one unit
UPDATE inventory
SET stock = stock - 3
WHERE id = 42 AND stock ≥ 3;
-- rowCount = 0 means the guard failed (not enough stock); no partial writ
```

Common mistakes & gotchas

- **Assuming multiple separate writes are atomic.** Two back-to-back `updateOne` / `UPDATE` calls without a transaction can partially fail, leaving inconsistent data. Wrap them or model them into one document/row.
- **Forgetting to pass the session/transaction handle.** In Mongo, a write without `{ session }` runs *outside* the transaction and is not rolled back. In pg, running on `pool` instead of the checked-out `client` escapes the BEGIN.

- **Running Mongo transactions on a standalone server.** Multi-document transactions require a replica set or sharded cluster; on a standalone `mongod` they error.
- **Check-then-act race conditions.** Reading a balance, then updating in a separate step, lets two concurrent requests both pass the check. Use a single conditional update or lock the row with `SELECT ... FOR UPDATE`.
- **Long-running transactions.** Holding a transaction open across slow work (HTTP calls, user think-time) holds locks and starves the database. Keep transactions short.
- **Not handling transient/write-conflict errors.** Under Serializable/high contention, transactions can be aborted by the database and must be *retried*. Not retrying surfaces spurious failures.
- **Using a transaction when one document/row would do.** Adds locking and overhead for no benefit; single-document/row writes are already atomic.

Best practices

- Reach for a transaction only when a single logical operation spans multiple rows/documents with an invariant between them; otherwise rely on built-in single-record atomicity.
- Keep transactions **short** — acquire, write, commit — and never do slow external I/O while one is open.
- Always pass the session/client to every operation inside the transaction, and clean up in `finally` (`release()` / `endSession()`).
- Prefer a single conditional atomic update (`$inc` with a guard, `UPDATE ... WHERE`) over read-then-write to avoid races cheaply.
- For genuine contention, lock explicitly (`SELECT ... FOR UPDATE`) or pick the right isolation level, and implement **retry** on transient conflicts.
- Choose the isolation level consciously: stronger levels prevent more anomalies at the cost of throughput.
- In MongoDB, model so that data changing together lives together — the best transaction is the one you did not need.

⭐ Interview lens

Atomicity (all operations commit or none do), Consistency (the DB moves between valid states honouring its constraints), Isolation (concurrent transactions do not see each other's uncommitted work), Durability (once committed, data survives crashes). The bank-transfer example ties them together: the debit and credit are atomic, the "sum stays constant" invariant is consistency, other transactions do not see a half-done transfer (isolation), and once committed it is on disk forever (durability).

Q: Does MongoDB support transactions?

Yes — single-document writes have always been atomic, and since 4.0 MongoDB supports multi-document ACID transactions via sessions (4.2 extended them to sharded clusters). The nuanced point: they require a replica set/cluster and carry overhead, and good document modeling often removes the need for them because related data lives in one atomically-updated document.

Q: What are isolation levels and what anomalies do they prevent?

They control how much concurrent transactions are shielded from each other. Read Uncommitted allows dirty reads; Read Committed prevents dirty reads; Repeatable Read also prevents non-repeatable reads; Serializable prevents phantom reads too and behaves as if transactions ran sequentially. Higher levels cost throughput. Naming an anomaly (dirty/non-repeatable/phantom read) and the level that stops it is the strong answer.

Q: When do you actually need a transaction?

When one logical operation spans multiple rows/documents and a partial result would be invalid — money transfers, order-plus-inventory, any cross-record invariant. When the change touches a single row/document, the database already guarantees atomicity, so a transaction is unnecessary overhead. The skill is recognising which case you are in.

Q: How do you avoid a race condition like overselling inventory?

Do not read-then-write in two steps — that lets two requests both pass the check. Use a single atomic conditional update: in Mongo `updateOne({ _id, stock: { $gte: qty } }, { $inc: { stock: -qty } })`, in SQL `UPDATE ... SET stock = stock - $1 WHERE id = $2 AND stock ≥ $1`, or lock the row with `SELECT ... FOR UPDATE` inside a transaction. The guard and the mutation must be one atomic operation.

Q: Why keep transactions short, and what is a write conflict?

An open transaction holds locks (or an MVCC snapshot) and blocks other work, so long transactions cause contention, lock waits, and blocked connections. Under high contention or Serializable isolation, the database may abort a transaction with a write-conflict/serialization error rather than allow an inconsistency — the correct response is to catch it and retry the whole transaction, which is why transaction code is often wrapped in a bounded retry loop.

Cheat-card

- **Transaction:** group of writes that all commit or all rollback.
- **ACID:** Atomicity, Consistency, Isolation, Durability.
- **SQL:** `BEGIN` → writes on the checked-out client → `COMMIT` / `ROLLBACK`.
- **Mongo:** `startSession` → `startTransaction` → pass `{ session }` → commit/abort; needs a replica set.
- **Single doc/row writes are already atomic** — often no transaction needed.
- **Avoid races** with one conditional atomic update or `SELECT ... FOR UPDATE`.
- **Isolation levels** trade throughput for fewer anomalies; retry on write conflicts; keep transactions short.

QUICK CHECK — no marks, just for you.

1. What do the four letters of ACID stand for?

- Availability, Consistency, Isolation, Durability
- Atomicity, Concurrency, Integrity, Distribution
- Access, Commit, Isolation, Delete

Check answer

2. Does MongoDB support transactions?

- No; MongoDB has no atomicity guarantees of any kind
- Yes; single-document writes are always atomic, and since 4.0 multi-document ACID transactions are available via sessions on a replica set or cluster
- Only single-document writes are atomic; multi-document transactions are impossible
- Yes, but only on a standalone mongod, never on a replica set

Check answer

3. How do you prevent a race condition like overselling inventory without a full transaction?

- Read the stock, then update it in a separate follow-up query
- Increase the connection pool size so requests do not overlap
- Use a single atomic conditional update where the guard and the mutation are one operation (e.g. updateOne with a $gte stock guard and $inc)
- Add an index on the stock column so reads are faster

Check answer

Previous
Connection Pooling

Next
Authentication Basics

LEARN MORE

ARTICLES & DOCS

- Transactions (PostgreSQL Tutorial, Section 3.4)
  www.postgresql.org
- START TRANSACTION, COMMIT, and ROLLBACK Statements (MySQL 8.0)
  dev.mysql.com
- ACID (Wikipedia)
  en.wikipedia.org
- Transactions — MongoDB Manual
  www.mongodb.com

VIDEOS



Relational Database ACID Transactions (Explained by Example)
Hussein Nasser

Hand-picked references for this lesson.